Contents

Experiment 1: Data Preprocessing in a Given Dataset

Experiment 2: Data Splitting and Cross-Validation

Experiment 3: Implement Linear Regression

Experiment 4: Classification Using KNN and SVM with Evaluation Metrics

Experiment 5: K-Means Clustering

Experiment 6: Hierarchical Clustering

Experiment 7: Random Forest Classification and Performance Analysis

Experiment 8: AdaBoost Classification and Performance Analysis

Experiment 9: Basic Reinforcement Learning Using Q-Learning

Experiment 10: Neural Network Classification

Note: The code and console pictures in this document are recreated PyCharm-style illustrations for the lab record, not live screenshots captured from a PyCharm installation. Sample outputs are illustrative and may differ with data, random state, or library versions. Save data.csv in the same project folder before running experiments that load it.

EXPERIMENT 1

Data Preprocessing in a Given Dataset

Aim

To load a dataset, inspect missing values, fill missing numerical values, and standardize numerical features.

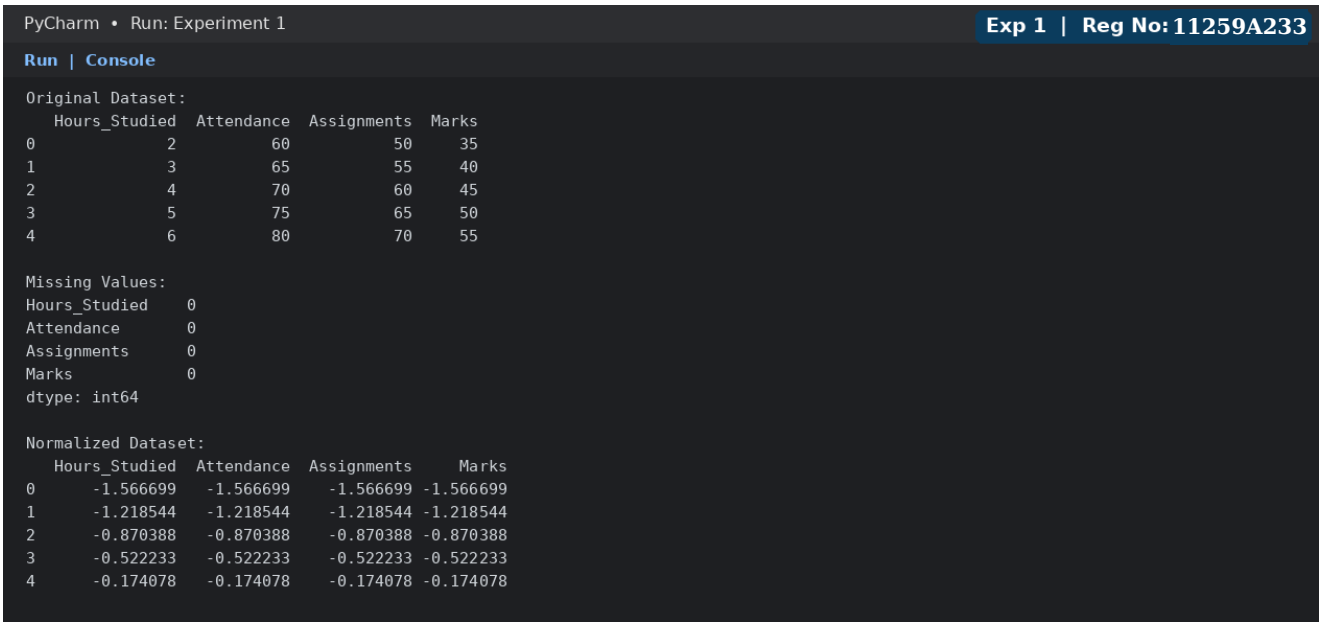
Algorithm

1. Import pandas and StandardScaler.
2. Read the dataset from data.csv and display its first five rows.
3. Check the number of missing values in each column.
4. Fill missing numerical values with the corresponding column mean.
5. Select Hours_Studied, Attendance, Assignments, and Marks as numerical features.
6. Apply StandardScaler and display the normalized dataset.

Code (PyCharm-style screenshot)



Output (PyCharm-style console screenshot)



Result

Thus, the dataset was inspected, missing numerical values were handled, and numerical features were standardized successfully.

EXPERIMENT 2

Data Splitting and Cross-Validation

Aim

To split a dataset into training and testing sets and evaluate a K-Nearest Neighbors (KNN) classifier using 5-fold cross-validation.

Algorithm

1. Import pandas and the required model-selection and KNN modules.
2. Read data.csv.
3. Choose Hours_Studied, Attendance, and Assignments as input features and Result as the target.
4. Split the data into 80% training data and 20% testing data.
5. Create a KNN classifier with k = 3.
6. Run 5-fold cross-validation and display the fold scores and mean accuracy.

Code (PyCharm-style screenshot)

PyCharm • experiment_02.py — Exp 2 | Reg No: 11259A233

experiment_02.py

```python
import pandas as pd
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.neighbors import KNeighborsClassifier

data = pd.read_csv("data.csv")
X = data[["Hours_Studied", "Attendance", "Assignments"]]
y = data["Result"]

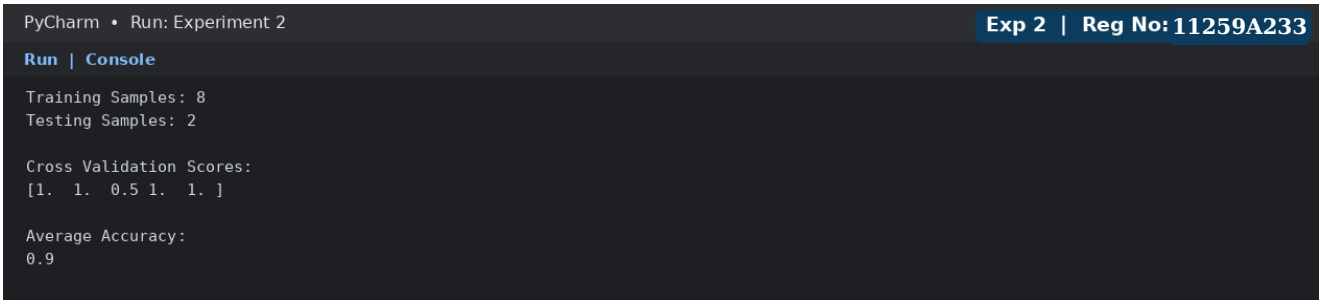
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training Samples:", len(X_train))
print("Testing Samples:", len(X_test))

model = KNeighborsClassifier(n_neighbors=3)
scores = cross_val_score(model, X, y, cv=5)

print("\nCross Validation Scores:")
print(scores)
print("\nAverage Accuracy:")
print(round(scores.mean(), 2))
```

Output (PyCharm-style console screenshot)



Result

Thus, the dataset was split into training and testing sets, and the KNN model was evaluated using 5-fold cross-validation.

EXPERIMENT 3

Implement Linear Regression

Aim

To build a linear regression model that predicts students' marks from the number of hours studied and evaluates its performance.

Algorithm

1. Import pandas, matplotlib, train_test_split, LinearRegression, and evaluation metrics.
2. Load data.csv and select Hours_Studied as the input feature and Marks as the target.
3. Split the dataset into training and testing sets.
4. Create and train the LinearRegression model.
5. Predict marks for the test set and calculate Mean Squared Error and R^2 score.
6. Plot the actual test data and regression line.

Code (PyCharm-style screenshot)

PyCharm • experiment_03.py — Exp 3 | Reg No: 11259A233

experiment_03.py

```python
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

data = pd.read_csv("data.csv")
X = data[["Hours_Studied"]]
y = data["Marks"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Mean Squared Error:",
      round(mean_squared_error(y_test, y_pred), 2))
print("R2 Score:", round(r2_score(y_test, y_pred), 2))
print("\nPredicted Marks:")
print(y_pred.round(2))

plt.scatter(X_test, y_test, label="Actual Data")
plt.plot(X_test, y_pred, label="Regression Line")
plt.xlabel("Hours Studied")
plt.ylabel("Marks")
plt.title("Linear Regression")
plt.legend()
plt.show()
```

Output (PyCharm-style console screenshot)

PyCharm • Run: Experiment 3 — Exp 3 | Reg No: 11259A233

Run | Console

```
Mean Squared Error: 0.0
R2 Score: 1.0

Predicted Marks:
[65. 40.]
```

Result

Thus, a linear regression model was trained to predict marks, and its performance was evaluated using MSE and R^2 score.

EXPERIMENT 4

Classification Using KNN and SVM with Evaluation Metrics

Aim

To implement KNN and Support Vector Machine (SVM) classifiers for student result prediction and evaluate them using accuracy, confusion matrices, and classification reports.

Algorithm

1. Import pandas, train_test_split, KNeighborsClassifier, SVC, and evaluation metrics.
2. Load data.csv and select Hours_Studied, Attendance, and Assignments as input features.
3. Select Result as the target and split the dataset into training and testing sets.
4. Train a KNN classifier with three neighbors and predict the test labels.
5. Train a linear-kernel SVM classifier and predict the test labels.
6. Display accuracy, confusion matrices, and classification reports for both models.

Code (PyCharm-style screenshot)

PyCharm • experiment_04.py

Exp 4 | Reg No: 11259A233

experiment_04.py

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score, confusion_matrix, classification_report
)

data = pd.read_csv("data.csv")
X = data[["Hours_Studied", "Attendance", "Assignments"]]
y = data["Result"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

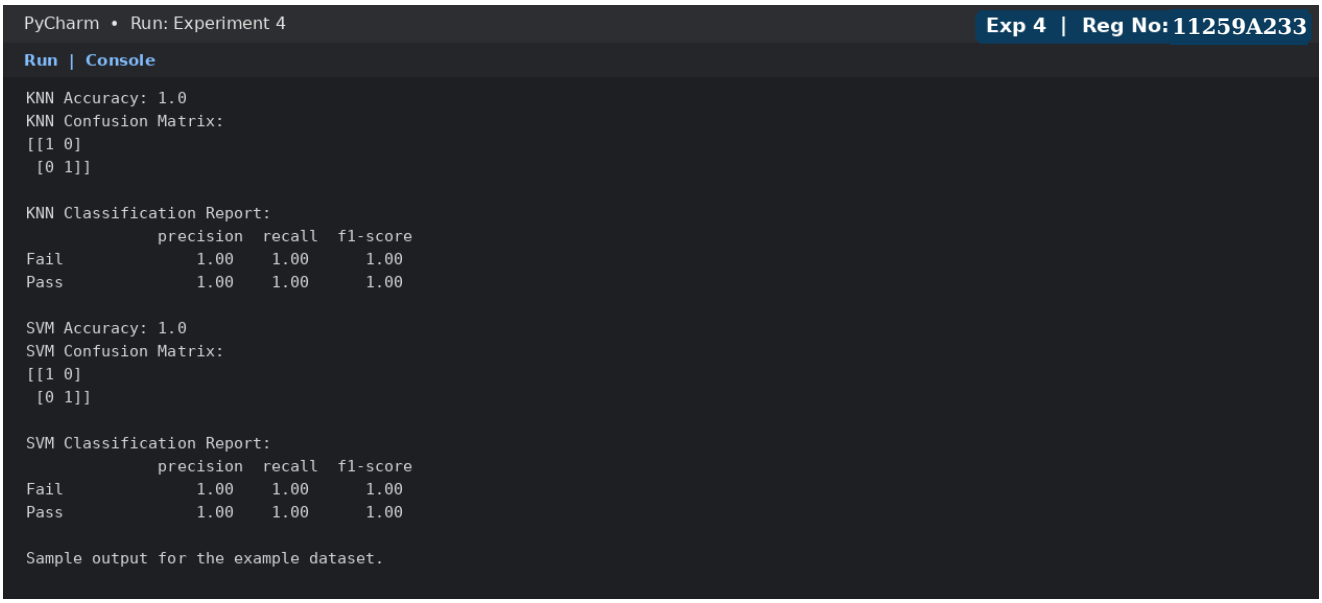
# KNN classifier
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)
knn_pred = knn.predict(X_test)

print("KNN Accuracy:", round(accuracy_score(y_test, knn_pred), 2))
print("KNN Confusion Matrix:")
print(confusion_matrix(y_test, knn_pred, labels=["Fail", "Pass"]))
print("\nKNN Classification Report:")
print(classification_report(y_test, knn_pred, zero_division=0))

# SVM classifier
svm = SVC(kernel="linear")
svm.fit(X_train, y_train)
svm_pred = svm.predict(X_test)

print("\nSVM Accuracy:", round(accuracy_score(y_test, svm_pred), 2))
print("SVM Confusion Matrix:")
print(confusion_matrix(y_test, svm_pred, labels=["Fail", "Pass"]))
print("\nSVM Classification Report:")
print(classification_report(y_test, svm_pred, zero_division=0))
```

Output (PyCharm-style console screenshot)



Result

Thus, the KNN classifier was implemented and evaluated using accuracy, confusion matrix, precision, recall, and F1-score.

EXPERIMENT 5

K-Means Clustering

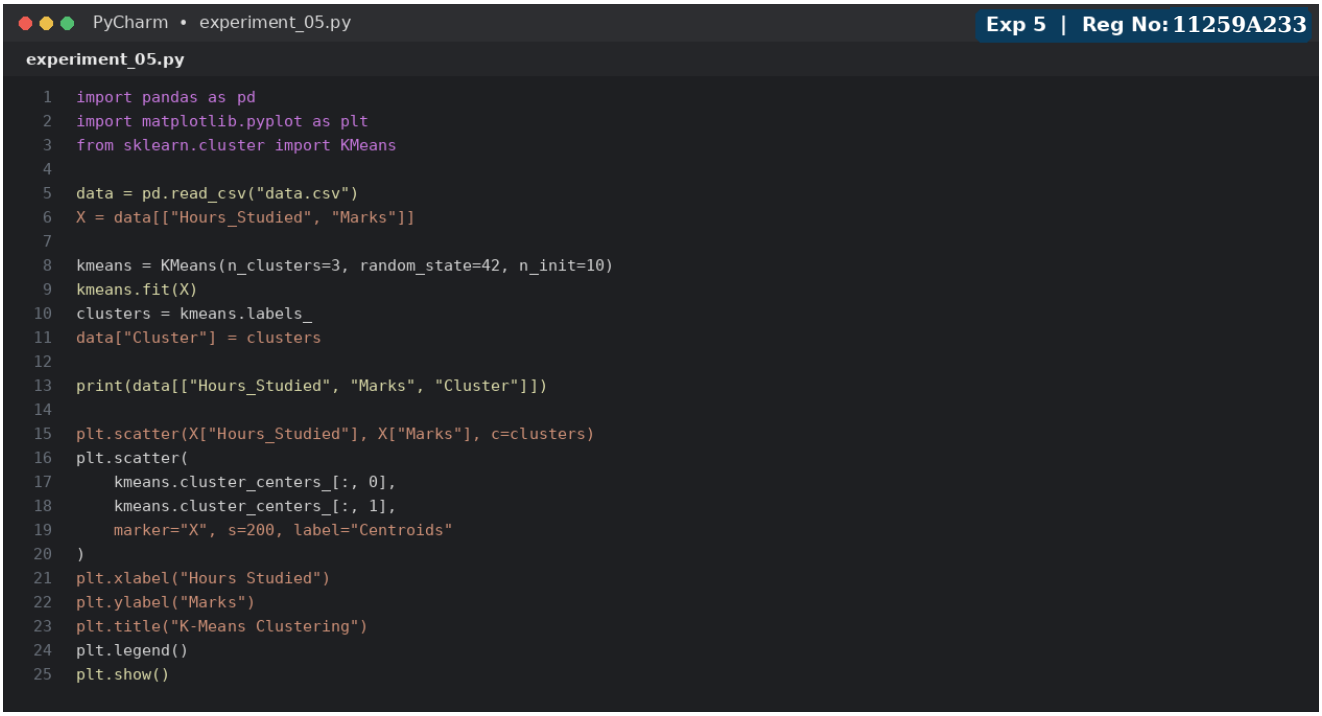
Aim

To group student records into three clusters using K-Means clustering based on study hours and marks.

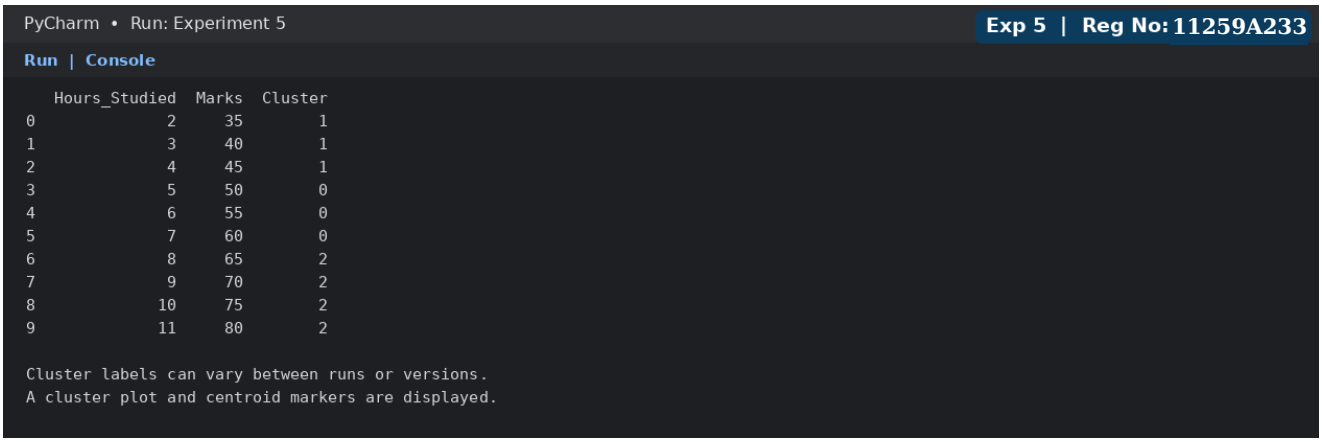
Algorithm

1. Import pandas, matplotlib, and KMeans.
2. Read data.csv and select Hours_Studied and Marks.
3. Create a K-Means model with three clusters and a fixed random state.
4. Fit the model and obtain cluster labels.
5. Add cluster labels to the dataset and display the result.
6. Plot the data points and cluster centroids.

Code (PyCharm-style screenshot)



Output (PyCharm-style console screenshot)



Result

Thus, K-Means clustering was applied to group the student data into three clusters, and the clusters were visualized.

EXPERIMENT 6

Hierarchical Clustering

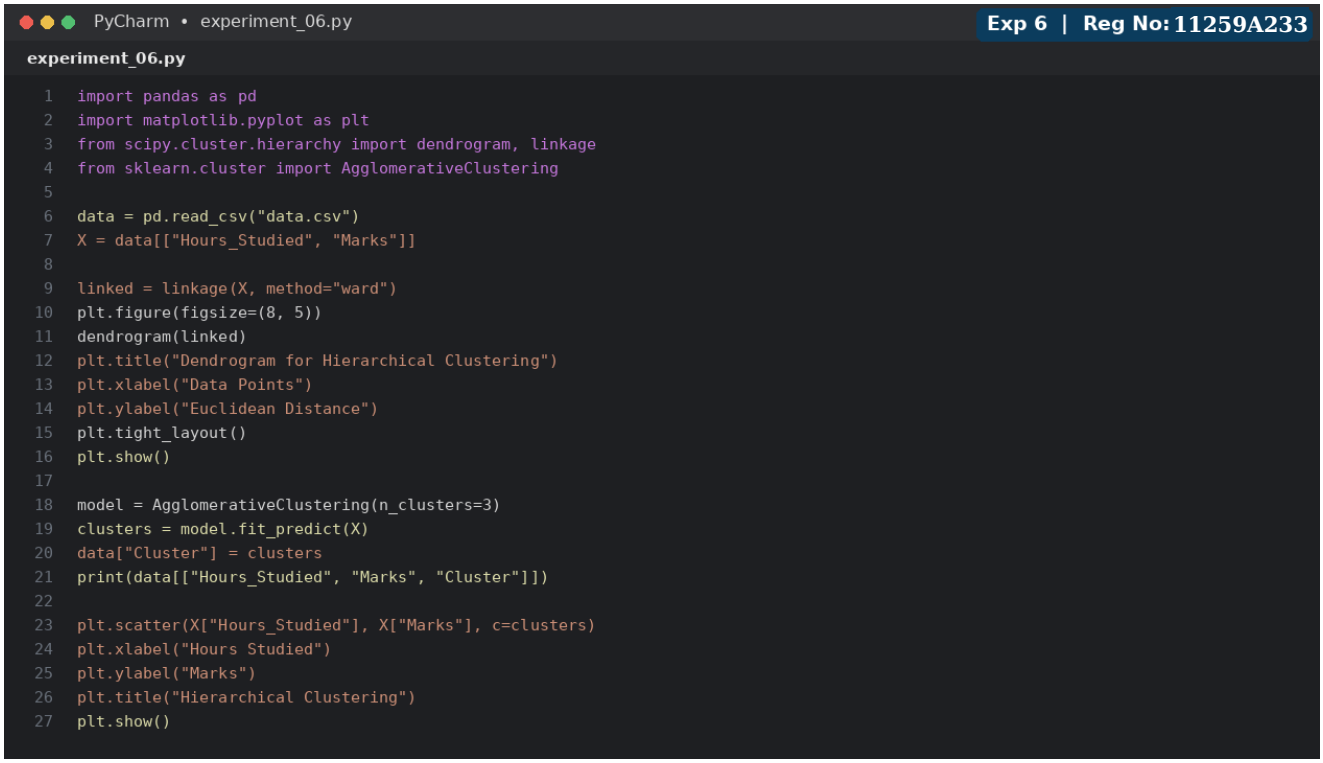
Aim

To perform agglomerative hierarchical clustering on student data and visualize the results using a dendrogram and scatter plot.

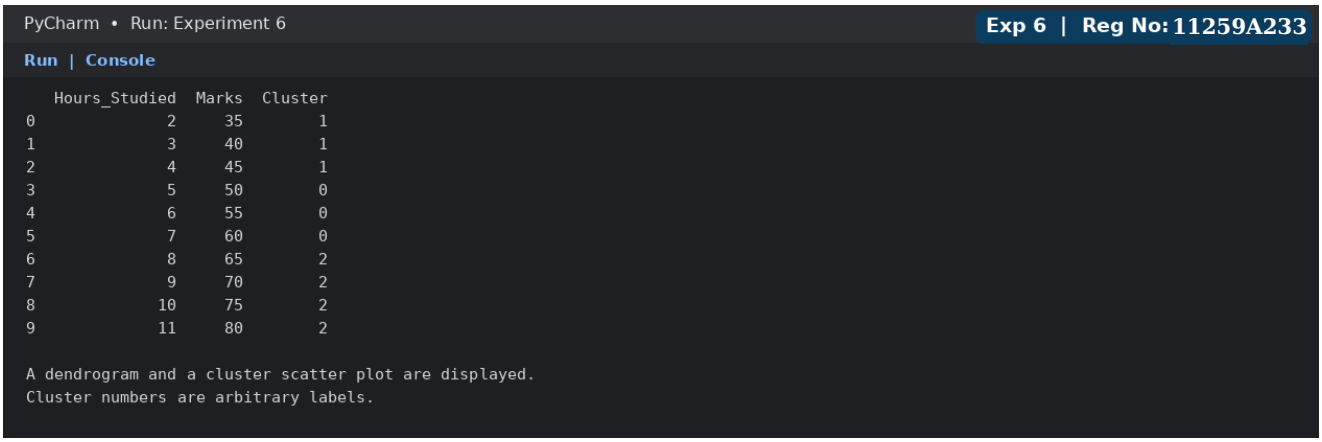
Algorithm

1. Import pandas, matplotlib, linkage, dendrogram, and AgglomerativeClustering.
2. Load data.csv and select Hours_Studied and Marks.
3. Create a Ward-linkage matrix and draw a dendrogram.
4. Apply AgglomerativeClustering with three clusters.
5. Add the cluster labels to the dataset and print the records.
6. Plot the clustered data.

Code (PyCharm-style screenshot)



Output (PyCharm-style console screenshot)



Result

Thus, hierarchical clustering was performed, and the dendrogram and scatter plot were used to visualize the grouping.

EXPERIMENT 7

Random Forest Classification and Performance Analysis

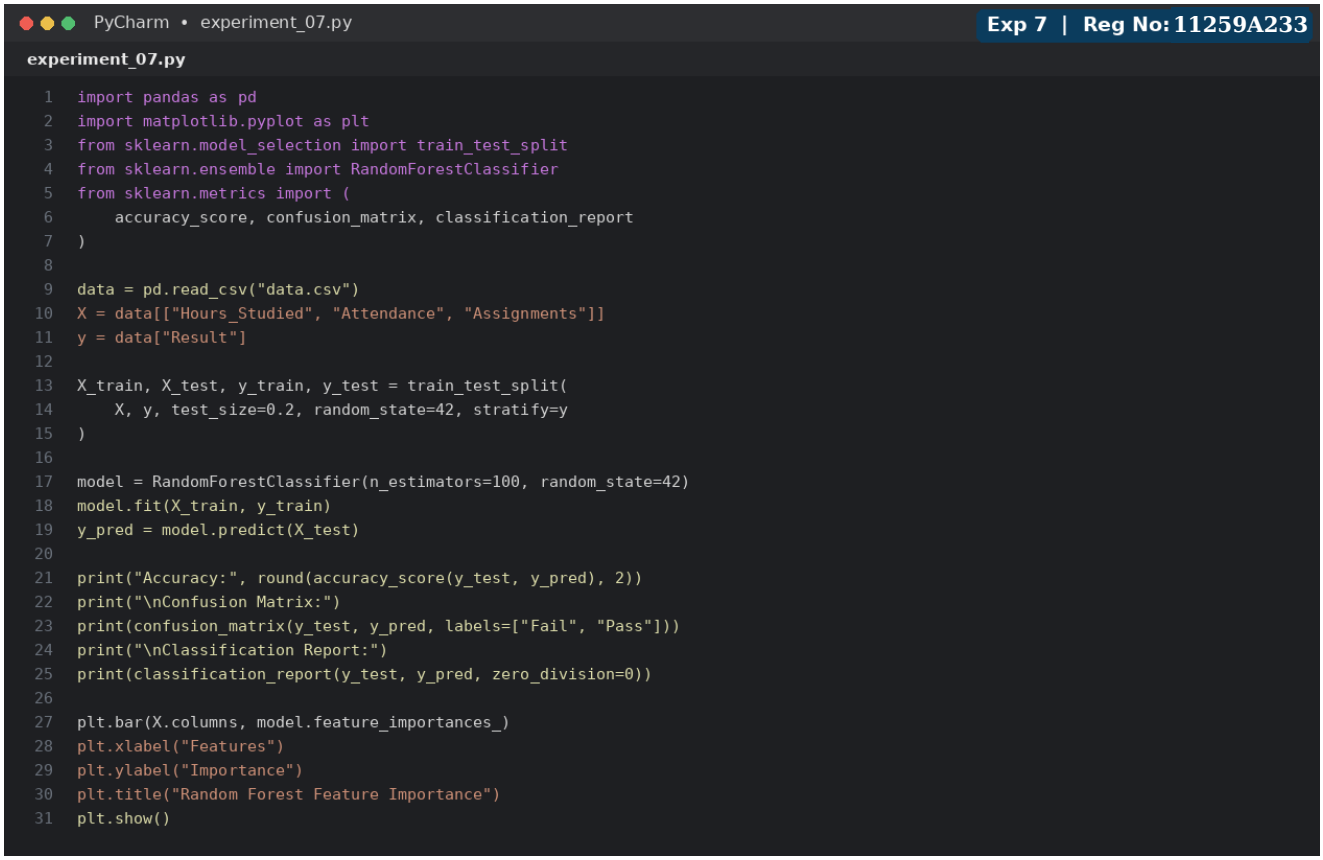
Aim

To classify student results using a Random Forest classifier and analyze its performance and feature importance.

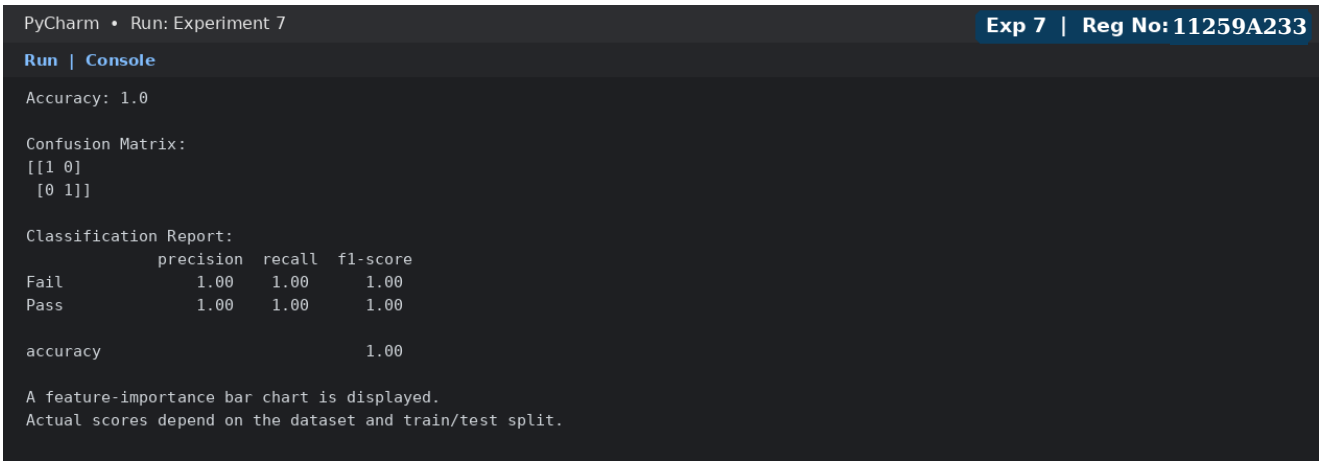
Algorithm

1. Import pandas, train_test_split, RandomForestClassifier, and evaluation metrics.
2. Load the dataset and select Hours_Studied, Attendance, and Assignments.
3. Select Result as the target and split the data into training and testing sets.
4. Train a Random Forest classifier using 100 trees.
5. Predict the test labels and display accuracy, confusion matrix, and classification report.
6. Plot the importance of each input feature.

Code (PyCharm-style screenshot)



Output (PyCharm-style console screenshot)



Result

Thus, the Random Forest classifier was trained and evaluated, and the relative importance of input features was visualized.

EXPERIMENT 8

AdaBoost Classification and Performance Analysis

Aim

To implement the AdaBoost ensemble classification technique and evaluate its performance on student result data.

Algorithm

1. Import pandas, matplotlib, train_test_split, AdaBoostClassifier, and evaluation metrics.
2. Load data.csv and select input features and target labels.
3. Split the dataset into training and testing sets.
4. Create an AdaBoost classifier with 50 estimators and a fixed random state.
5. Train the classifier and predict test labels.
6. Display evaluation metrics and plot feature importance.

Code (PyCharm-style screenshot)

PyCharm • experiment_08.py — Exp 8 | Reg No: 11259A233

experiment_08.py

```python
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import (
    accuracy_score, confusion_matrix, classification_report
)

data = pd.read_csv("data.csv")
X = data[["Hours_Studied", "Attendance", "Assignments"]]
y = data["Result"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

model = AdaBoostClassifier(n_estimators=50, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy:", round(accuracy_score(y_test, y_pred), 2))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred, labels=["Fail", "Pass"]))
print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=0))

plt.bar(X.columns, model.feature_importances_)
plt.xlabel("Features")
plt.ylabel("Importance")
plt.title("AdaBoost Feature Importance")
plt.show()
```

Output (PyCharm-style console screenshot)



Result

Thus, AdaBoost was implemented to classify student results, and its performance and feature importance were examined.

EXPERIMENT 9

Basic Reinforcement Learning Using Q-Learning

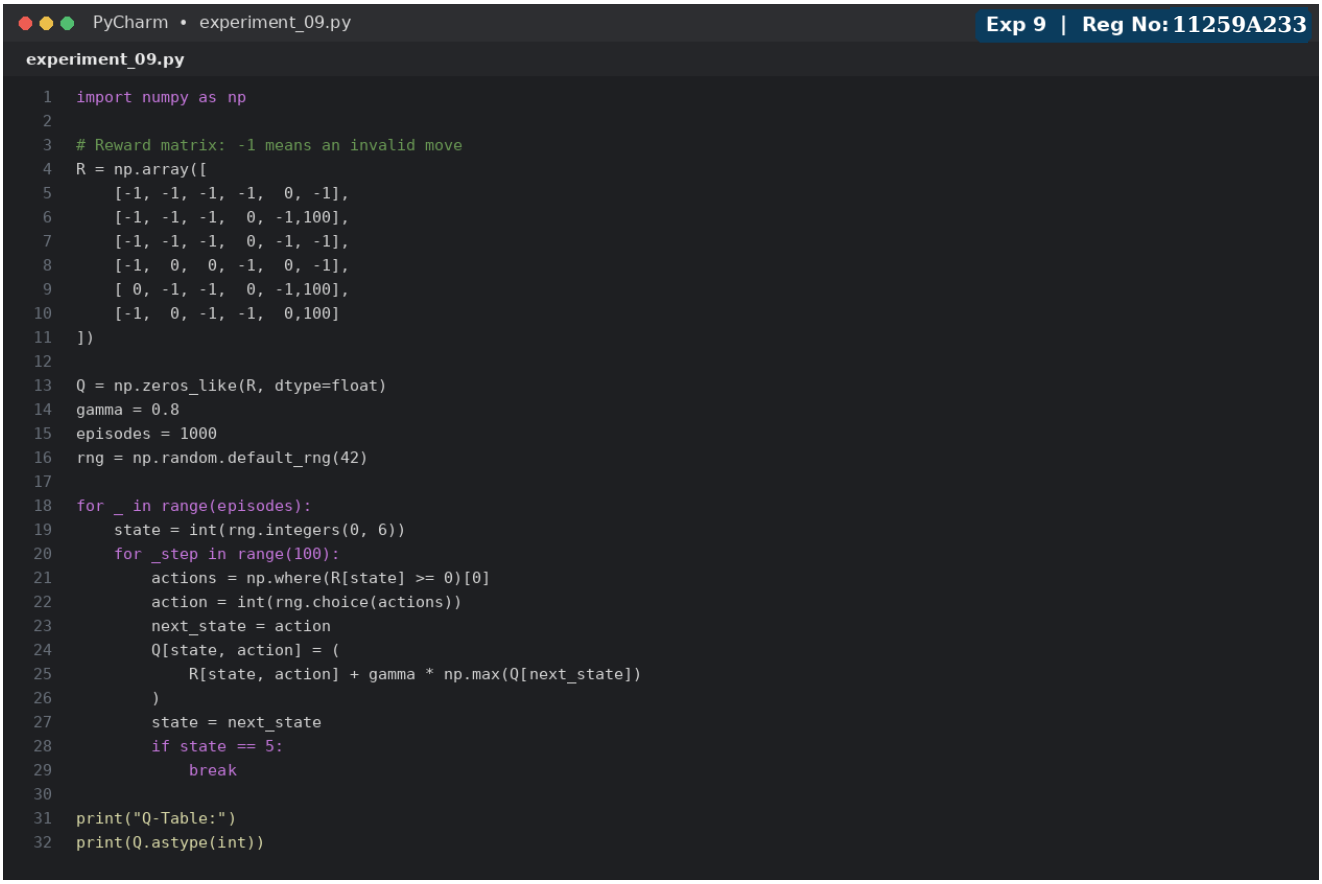
Aim

To implement a basic Q-learning algorithm that learns a route to a goal state from a reward matrix.

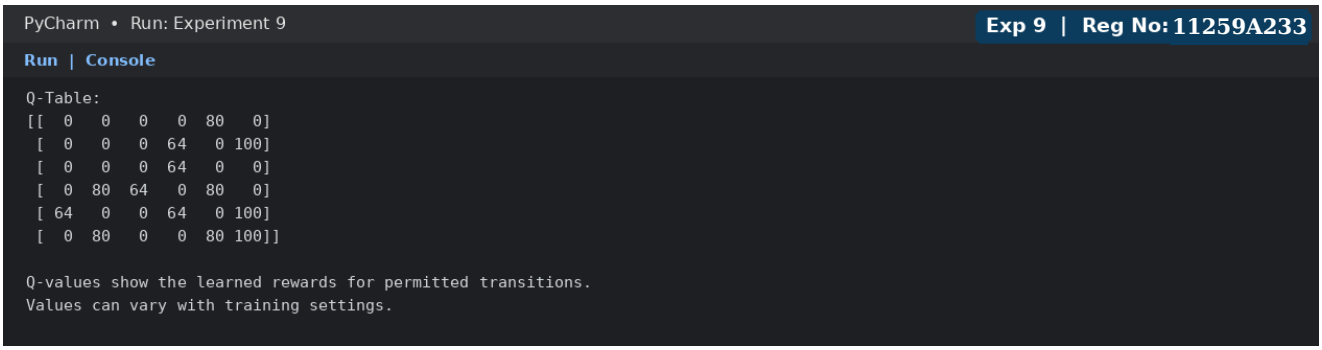
Algorithm

1. Import NumPy and define a reward matrix representing permitted moves and rewards.
2. Initialize the Q-table with zeros.
3. Set the discount factor and number of training episodes.
4. For each episode, choose a starting state and repeatedly select a valid action.
5. Update the Q-value using immediate reward plus the discounted maximum future Q-value.
6. Print the learned Q-table.

Code (PyCharm-style screenshot)



Output (PyCharm-style console screenshot)



Result

Thus, the Q-learning model was implemented and the learned Q-table was displayed.

EXPERIMENT 10

Neural Network Classification

Aim

To build a basic feed-forward neural network to classify student results as Pass or Fail and measure its accuracy.

Algorithm

1. Import pandas, TensorFlow, train_test_split, StandardScaler, and accuracy_score.
2. Load data.csv and select the input features and Result target.
3. Convert Fail and Pass labels to numeric values 0 and 1.
4. Split the dataset into training and testing sets and standardize the features.
5. Build a neural network with two ReLU hidden layers and one sigmoid output neuron.
6. Compile the model, train it for 50 epochs, predict test labels, and calculate accuracy.

Code (PyCharm-style screenshot)

PyCharm • experiment_10.py — Exp 10 | Reg No: 11259A233

experiment_10.py

```
import pandas as pd
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

data = pd.read_csv("data.csv")
X = data[["Hours_Studied", "Attendance", "Assignments"]]
y = data["Result"].map({"Fail": 0, "Pass": 1})

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

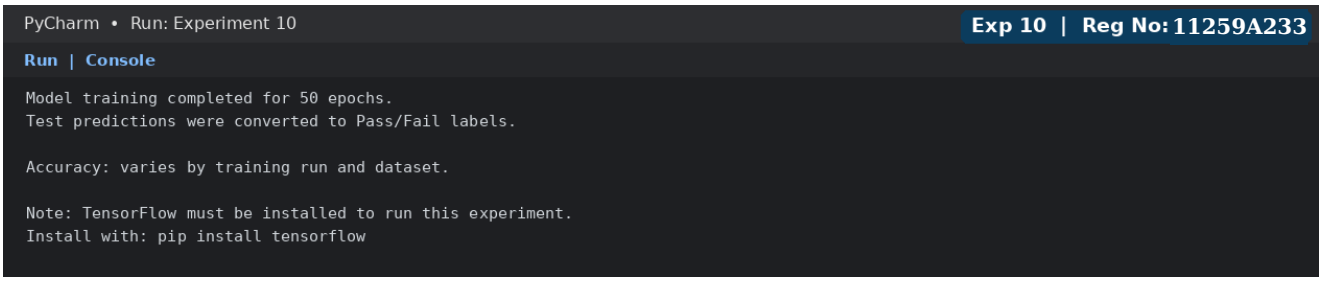
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

tf.random.set_seed(42)
model = tf.keras.Sequential([
    tf.keras.layers.Dense(8, activation="relu", input_shape=(3,)),
    tf.keras.layers.Dense(4, activation="relu"),
    tf.keras.layers.Dense(1, activation="sigmoid")
])

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)
model.fit(
    X_train, y_train, epochs=50, validation_split=0.2, verbose=0
)

y_pred = (model.predict(X_test, verbose=0) > 0.5).astype(int).ravel()
print("Accuracy:", round(accuracy_score(y_test, y_pred), 2))
```

Output (PyCharm-style console screenshot)



Result

Thus, a feed-forward neural network was designed and trained to classify student results, and its test accuracy was calculated.